

TECHNICAL STANDARD OPERATING PROCEDURE

PostgreSQL 18

High Availability (HA) and Disaster Recovery (DR)

Architecture Features, Replication, Failover, Backup and Recovery

SOURCE CONTROL STATEMENT

Technical content in this SOP is derived exclusively from PostgreSQL Official Documentation webpages on postgresql.org. No third-party documentation is used as a technical source.

Reference release: PostgreSQL 18

Document version: 1.0

Date: 26 August 2026

Classification: Technical Reference / Operations SOP

Document Control

Field	Value
Document Title	PostgreSQL 18 - High Availability and Disaster Recovery Technical SOP
Document Type	Technical Standard Operating Procedure / Architecture Reference
PostgreSQL Release	PostgreSQL 18
Version	1.0
Issue Date	26 August 2026
Technical Source Policy	PostgreSQL Official Documentation only (postgresql.org)
Primary Documentation Areas	Chapter 25 - Backup and Restore; Chapter 26 - High Availability, Load Balancing, and Replication; Chapter 29 - Logical Replication; PostgreSQL 18 server configuration and system catalog documentation

Purpose

This SOP provides a controlled technical reference for PostgreSQL 18 high availability and disaster recovery capabilities. It explains native PostgreSQL replication, standby, promotion, WAL archiving, backup, PITR, logical replication failover, monitoring, and the limits of what PostgreSQL itself provides for automated HA.

Scope

- PostgreSQL 18 physical replication and standby operation.
- Synchronous and asynchronous replication behavior.
- Hot standby and cascading replication.
- Replication slots and WAL-retention risks.
- Standby promotion, failover, switchover concepts, and split-brain prevention requirements.
- WAL archiving, base backups, incremental backups, pg_combinebackup, and PITR.
- Logical replication and logical replication failover.
- Shared-disk and file-system replication options described by PostgreSQL documentation.
- Operational monitoring queries and failure scenarios derived from PostgreSQL capabilities.

Source and Validation Rule

Exclusive source policy: Every technical statement in this SOP is based on PostgreSQL Official Documentation. Where PostgreSQL describes an external requirement (for example, failure detection, fencing, or client routing), this SOP states that requirement without presenting any third-party product as an authoritative source.

Document Index / Table of Contents

1. HA and DR Fundamentals

- 1.1 High Availability
- 1.2 Disaster Recovery
- 1.3 HA vs DR
- 1.4 RPO and RTO

2. PostgreSQL HA/DR Feature Matrix

- 2.1 Native and external responsibilities

3. Physical Streaming Replication

- 3.1 Architecture
- 3.2 Advantages and limitations

4. Asynchronous and Synchronous Replication

- 4.1 Asynchronous replication
- 4.2 Synchronous replication
- 4.3 FIRST and ANY
- 4.4 synchronous_commit

5. Standby Operation

- 5.1 Warm standby
- 5.2 Hot standby
- 5.3 Recovery conflicts
- 5.4 Cascading replication

6. Replication Slots and WAL Retention

- 6.1 Physical/logical slots
- 6.2 max_slot_wal_keep_size
- 6.3 idle_replication_slot_timeout
- 6.4 Monitoring

7. Promotion, Failover, Switchover and Split Brain

- 7.1 Promotion
- 7.2 standby.signal
- 7.3 Failover
- 7.4 Switchover
- 7.5 Fencing / STONITH

8. WAL Archiving, Backup and PITR

- 8.1 WAL log shipping

- 8.2 Continuous archiving
- 8.3 Base backup
- 8.4 Incremental backup
- 8.5 pg_combinebackup
- 8.6 PITR

9. Logical Replication and Failover

- 9.1 Physical vs logical
- 9.2 DDL and sequence limitations
- 9.3 failover slots
- 9.4 sync_replication_slots

10. Alternative HA Architectures

- 10.1 Shared-disk failover
- 10.2 File-system/block replication

11. Monitoring and Operational Checks

- 11.1 pg_stat_replication
- 11.2 recovery state
- 11.3 LSN functions
- 11.4 replication-slot checks

12. Failure Scenarios and Resiliency Model

- 12.1 Primary failure
- 12.2 Site failure
- 12.3 Accidental DROP
- 12.4 Slot-driven WAL growth
- 12.5 Hot-standby query conflict

13. Production Design Principles

- 13.1 Layered resiliency
- 13.2 HA is not backup
- 13.3 Native PostgreSQL vs external HA responsibilities

Appendix A. Official PostgreSQL References

1. HA and DR Fundamentals

1.1 High Availability (HA)

High Availability is an architecture in which database service can continue or be restored quickly when a PostgreSQL server, host, operating system, storage component, or related service path fails. PostgreSQL provides standby-server and promotion mechanisms that are commonly used as building blocks for HA.



Important: PostgreSQL provides replication and standby promotion, but PostgreSQL documentation explicitly states that it does not provide the system software required to identify primary failure and notify the standby automatically.

Official PostgreSQL source: [PostgreSQL 18 - Failover](#)

1.2 Disaster Recovery (DR)

Disaster recovery addresses recovery after failures that exceed the local HA boundary, including loss of a primary site or loss/corruption of the active database copy. PostgreSQL DR mechanisms include physical standbys, continuous WAL archiving, base backups, incremental backups, and point-in-time recovery.

Official PostgreSQL source: [PostgreSQL 18 - Continuous Archiving and PITR](#)

1.3 HA vs DR

Area	High Availability	Disaster Recovery
Primary goal	Maintain or rapidly restore service	Recover service/data after major failure
Typical PostgreSQL mechanisms	Physical standby, streaming replication, synchronous replication, promotion	Remote standby, WAL archive, base/incremental backup, PITR
Logical/user-error recovery	Replication normally replays the same change	PITR can recover to a selected earlier target when required WAL is available

Area	High Availability	Disaster Recovery
Automatic cluster orchestration	Not supplied as a complete PostgreSQL feature	Not supplied as a complete PostgreSQL feature

1.4 RPO and RTO

Recovery Point Objective (RPO) is a business recovery target describing how much committed data loss is acceptable after a failure. Recovery Time Objective (RTO) describes how long service may remain unavailable. PostgreSQL features support different durability and recovery designs, but RPO/RTO values are architecture and business requirements rather than fixed PostgreSQL guarantees.

Precision rule: Do not assign a fixed RPO such as “5 seconds” to asynchronous replication. Actual loss at failover depends on replication state and failure timing.

2. PostgreSQL HA/DR Feature Matrix

Feature / Option	HA	DR	PostgreSQL Native?	Technical Purpose
Physical streaming replication	Yes	Yes	Yes	Maintain a physical standby from WAL.
Asynchronous replication	Yes	Yes	Yes	Lower commit dependency on standby; acknowledged transactions can be absent from standby at failure.
Synchronous replication	Yes	Possible	Yes	Wait for required synchronous standby responses according to configuration.
Warm standby	Yes	Supporting	Yes	Recovery standby not serving normal read-only queries.
Hot standby	Yes	Supporting	Yes	Read-only queries while recovery continues.
Cascading replication	Yes	Yes	Yes	Standby streams WAL to downstream standby.
Replication slots	Supporting	Supporting	Yes	Retain WAL/resources required by replication consumers.
Standby promotion	Yes	Yes	Yes	Convert standby to normal primary operation.
WAL archiving	Supporting	Yes	Yes	Archive WAL for recovery/log shipping.
Base backup	Supporting	Yes	Yes	Physical backup foundation for recovery/standby creation.
Incremental backup	No	Yes	Yes	Back up changed relation blocks using WAL summaries.

Feature / Option	HA	DR	PostgreSQL Native?	Technical Purpose
PITR	No	Yes	Yes	Recover to a configured recovery target using base backup + WAL.
Logical replication	Limited	Yes / migration	Yes	Publication/subscription based selective replication.
Logical replication failover	Supporting	Supporting	Yes	Synchronize logical failover slots to physical standby.
Shared-disk failover	Yes	Limited	Architecture documented	Rapid failover using one shared database copy.
File-system replication	Yes	Possible	External technology	Replication below PostgreSQL at storage/file-system layer.
Automatic failure detection / fencing / client routing	Required for automated HA	Context dependent	No complete native cluster manager	Must be supplied by surrounding system/software.

Official PostgreSQL source: [PostgreSQL 18 - Comparison of Different Solutions](#)

3. Physical Streaming Replication

3.1 Architecture and Behavior

PostgreSQL physical standby servers maintain a copy of the database cluster by replaying WAL records generated by the primary. Streaming replication allows a standby to connect to a WAL sender and receive WAL records as they are generated, rather than relying only on completed archived WAL files.

```

Primary PostgreSQL
  |
  | WAL stream
  v
Physical Standby
  |
  |-- replay WAL during recovery
  |-- may be promoted when required

```

A standby can obtain WAL from an archive, from `pg_wal` where available, and through streaming replication according to recovery configuration.

Official PostgreSQL source: [PostgreSQL 18 - Log-Shipping Standby Servers](https://www.postgresql.org/docs/18/log-ship.html)

3.2 Advantages and Limitations

Advantages	Limitations / Operational Considerations
Built into PostgreSQL.	Physical replication applies to the PostgreSQL cluster rather than a selected set of tables.
Supports synchronous and asynchronous configurations.	Replication lag and WAL availability must be monitored.
Supports hot standby and cascading replication.	Asynchronous failover can lose transactions not yet received by the standby.
Standby can be promoted.	Automatic failure detection and full cluster failover orchestration are not supplied by PostgreSQL itself.
Can be combined with WAL archive recovery.	Primary and standby release compatibility must follow PostgreSQL guidance; keeping systems at the same release level is recommended.

4. Asynchronous and Synchronous Replication

4.1 Asynchronous Replication

In asynchronous streaming replication, a commit can be acknowledged on the primary before the corresponding WAL has been durably received by a standby. Therefore, transactions committed shortly before a primary failure might not be present on the standby selected for failover.

```
Client COMMIT
  |
  v
Primary acknowledges commit
  |
+----- WAL -----> Asynchronous standby
```

RPO note: PostgreSQL does not define a fixed time-based RPO for asynchronous replication. Potential loss depends on the current replication state, workload, network, standby performance, and the moment of failure.

Official PostgreSQL source: [PostgreSQL 18 - High Availability, Load Balancing, and Replication](#)

4.2 Synchronous Replication

Synchronous replication can require transaction commit processing to wait for one or more synchronous standbys. The exact durability and visibility guarantee also depends on `synchronous_commit`.

Official PostgreSQL source: [PostgreSQL 18 - Replication Configuration](#)

4.3 FIRST and ANY Synchronous Standby Selection

PostgreSQL supports priority-based and quorum-based synchronous standby selection through `synchronous_standby_names`.

```
# Priority method
synchronous_standby_names = 'FIRST 1 (standby1, standby2) '

# Quorum method
synchronous_standby_names = 'ANY 2 (standby1, standby2, standby3) '
```

Method	Meaning
FIRST n	Wait for n highest-priority eligible synchronous standbys from the listed set.
ANY n	Wait for responses from any n eligible standbys in the listed set.

4.4 synchronous_commit

Value	Commit waits for / durability characteristic
remote_apply	Waits until commit is applied on synchronous standby; changes are visible to queries there.
on	Waits until WAL is flushed to durable storage on required synchronous standbys.
remote_write	Waits until WAL is written to standby operating-system storage, not necessarily durably flushed against an OS crash.
local	Waits for local WAL flush on primary; does not wait for remote replication.
off	Does not wait for local WAL flush; a recent acknowledged transaction can be lost after a server crash.

Correct RPO wording: A correctly designed synchronous configuration can provide an effective RPO of 0 for acknowledged transactions within the failure scope covered by the synchronous durability configuration. Avoid the unconditional statement “synchronous replication always guarantees zero data loss.”

Official PostgreSQL source: [PostgreSQL 18 - WAL Configuration / synchronous_commit](#)

5. Standby Operation

5.1 Warm Standby

PostgreSQL uses the term warm standby for a standby that is kept current for failover but is not available for normal read-only queries during recovery. It can be promoted when required.

Official PostgreSQL source: [PostgreSQL 18 - High Availability](#)

5.2 Hot Standby

Hot standby allows read-only connections and queries while WAL recovery continues. This permits standby hardware to support read-only workloads while remaining available as a recovery target.

```

Primary (Read/Write)
  | WAL
  v
Hot Standby (Read Only)
  ^
  | reporting / read-only clients

```

Official PostgreSQL source: [PostgreSQL 18 - Hot Standby](#)

5.3 Hot Standby Recovery Conflicts

Standby queries can conflict with WAL replay. PostgreSQL may delay WAL application for configured periods and can cancel conflicting queries so recovery can continue.

`hot_standby_feedback` can reduce certain cleanup conflicts by sending feedback to the primary, but it can also contribute to table bloat on the primary.

```
max_standby_streaming_delay
max_standby_archive_delay
hot_standby_feedback
```

Official PostgreSQL source: [PostgreSQL 18 - Hot Standby](#)

5.4 Cascading Replication

A cascading standby receives WAL from an upstream server and can simultaneously send WAL to downstream standbys. This can reduce the number of direct replication connections to the primary and support multi-level topologies.

```
Primary
|
v
Standby 1
| \
|  \----> Standby 3
v
Standby 2
```

Synchronous design note: Downstream cascading standbys are not directly connected to the primary. Synchronous standby selection on the primary operates on directly connected standby connections.

Official PostgreSQL source: [PostgreSQL 18 - Log-Shipping Standby Servers](#)

6. Replication Slots and WAL Retention

6.1 Purpose and Risk

Replication slots retain WAL and, for logical decoding, other resources needed by replication consumers. Slots persist independently of consumer health. If a slot remains while its consumer is inactive, required WAL can accumulate and consume substantial storage.

```
SELECT pg_create_physical_replication_slot('standby1');

SELECT * FROM pg_replication_slots;
```

Critical operational risk: A replication slot that is no longer consumed can retain WAL until storage pressure becomes severe. PostgreSQL documentation explicitly warns that slots know nothing about consumer state and can retain required resources.

Official PostgreSQL source: [PostgreSQL 18 - Logical Decoding Concepts / Replication Slots](#)

6.2 max_slot_wal_keep_size

max_slot_wal_keep_size limits the amount of WAL that replication slots are allowed to retain in pg_wal at checkpoint time. The default value -1 allows unlimited retention. If a finite limit is exceeded and required WAL is removed, a lagging consumer may no longer be able to continue from the slot.

Official PostgreSQL source: [PostgreSQL 18 - Replication Configuration](#)

6.3 idle_replication_slot_timeout

idle_replication_slot_timeout can invalidate replication slots that remain inactive longer than the configured time. The default value 0 disables the timeout. The parameter is intended to help manage abandoned or idle slots, but its use must match operational requirements.

Official PostgreSQL source: [PostgreSQL 18 - Replication Configuration](#)

6.4 Recommended Monitoring Fields

```
SELECT
    slot_name,
    slot_type,
    active,
    inactive_since,
    restart_lsn,
    confirmed_flush_lsn,
    wal_status,
    safe_wal_size,
    invalidation_reason
FROM pg_replication_slots;
```

PostgreSQL 18 exposes WAL availability states for slots including reserved, extended, unreserved, and lost. These fields help DBAs identify retention pressure and invalidated slots.

Official PostgreSQL source: [PostgreSQL 18 - pg_replication_slots](#)

7. Promotion, Failover, Switchover and Split Brain

7.1 Standby Promotion

A standby can be promoted so that it leaves recovery and begins normal read/write operation as a primary.

```
pg_ctl promote

-- SQL alternative
SELECT pg_promote();
```

Official PostgreSQL source: [PostgreSQL 18 - Failover](#)

7.2 standby.signal

Modern PostgreSQL uses standby.signal to enter standby mode. recovery.conf is no longer used; recovery settings are normal server configuration parameters.

Official PostgreSQL source: [PostgreSQL 18 - Recovery Target Settings](#)

7.3 Failover

Failover is the operational transition to a standby after the current primary becomes unavailable. PostgreSQL supplies promotion mechanisms, but the surrounding HA system must decide that the primary has failed and coordinate the transition.

```
Detect primary failure
    |
Prevent old primary from serving writes
    |
Promote standby
    |
Redirect clients / service endpoint
```

7.4 Switchover

A planned role change between primary and standby is commonly referred to operationally as a switchover. PostgreSQL documentation recommends regular switching/testing of standby servers so that failover mechanisms and operational procedures are known to work.

Official PostgreSQL source: [PostgreSQL 18 - Failover](#)

7.5 Split Brain and Fencing / STONITH

After a standby is promoted, the old primary must not later rejoin and continue accepting writes as another primary. PostgreSQL documentation specifically notes that a mechanism is needed to inform or prevent the old primary from continuing operation; one example is STONITH (fencing).

```

Network / primary failure
|
Standby promoted
|
Old primary returns without fencing
|
Two writable primaries -> divergent data risk

```

Mandatory HA design concern: A complete automated HA design must prevent an old primary from becoming writable after another node has been promoted.

Official PostgreSQL source: [PostgreSQL 18 - Failover](#)

8. WAL Archiving, Backup and Point-in-Time Recovery

8.1 WAL Log Shipping

PostgreSQL standby servers can recover by reading archived WAL segments. WAL log shipping can operate independently or together with streaming replication, providing another path for a standby to obtain required WAL.

Official PostgreSQL source: [PostgreSQL 18 - Log-Shipping Standby Servers](#)

8.2 Continuous WAL Archiving

PostgreSQL maintains WAL in the `pg_wal` subdirectory of the cluster data directory. By archiving completed WAL segments and combining them with a physical backup, PostgreSQL can perform continuous archive recovery and point-in-time recovery.

```

PostgreSQL data changes
|
v
pg_wal
| archive
v
WAL Archive
+
Physical Base Backup
|
v
Recovery / PITR

```

Official PostgreSQL source: [PostgreSQL 18 - Continuous Archiving and PITR](#)

8.3 Base Backup

A physical base backup provides a file-system-level starting point for continuous recovery and for creating physical standby systems. `pg_basebackup` is PostgreSQL's supplied utility for taking base backups from a running cluster.

```
pg_basebackup [options]
```

Official PostgreSQL source: [PostgreSQL 18 - pg_basebackup](#)

8.4 Incremental Backup

PostgreSQL 18 supports incremental base backups through `pg_basebackup --incremental`. The command receives the backup manifest from an earlier backup on the same server. PostgreSQL uses WAL summaries stored under `pg_wal/summaries` to identify relation blocks changed since the earlier backup.

```
pg_basebackup --incremental=/path/to/previous/backup_manifest ...
```

Incremental backups still require the earlier backups on which they depend, and the normal WAL requirements for continuous recovery still apply. PostgreSQL does not automatically manage dependency retention for incremental backup chains.

Official PostgreSQL source: [PostgreSQL 18 - Continuous Archiving / Incremental Backup](#)

8.5 pg_combinebackup

`pg_combinebackup` reconstructs a synthetic full backup from a full backup plus the dependent incremental backups. Required backups are supplied from oldest to newest. If a required predecessor backup is missing, the incremental backup cannot be reconstructed correctly.

```
pg_combinebackup -o /restore/full /backup/full /backup/inc1 /backup/inc2
```

Official PostgreSQL source: [PostgreSQL 18 - pg_combinebackup](#)

8.6 Point-in-Time Recovery (PITR)

PITR restores a physical backup and replays archived WAL until a configured recovery target is reached. It is a recovery mechanism for situations such as an unwanted data change, provided the necessary backup and complete required WAL history are available.

```
Base / reconstructed full backup
    +
Required archived WAL
    |
    v
Recovery to configured target
```

Recovery targets can include time, transaction ID, restore point name, or LSN depending on the recovery configuration. Recovery is constrained by the backup and WAL history available; “recover to any time” is therefore not an unconditional guarantee.

Official PostgreSQL source: [PostgreSQL 18 - Continuous Archiving and PITR](#)

9. Logical Replication and Logical Replication Failover

9.1 Logical Replication

PostgreSQL logical replication uses a publish/subscribe model. A publisher exposes changes through publications and a subscriber receives those changes through subscriptions. This permits replication of selected tables and is useful for data distribution and migration scenarios.

```
Publisher
  | Publication
  v
Subscriber / Subscription
```

Official PostgreSQL source: [PostgreSQL 18 - Logical Replication](#)

9.2 Physical vs Logical Replication

Area	Physical Replication	Logical Replication
Replication scope	Physical PostgreSQL cluster state	Published logical table changes
Selected tables	No	Yes
Standby/subscriber write model	Physical standby remains in recovery until promotion	Subscriber is an independent PostgreSQL database and can contain other writable objects
DDL/schema replication	Physical WAL reproduces physical database changes	Schema definitions / DDL are not automatically replicated by normal logical replication
Sequence state	Physical state is part of physical replication	Sequence state is not automatically replicated
Typical use	HA / physical DR	Selective replication, integration, migrations

9.3 DDL and Sequence Limitations

Logical replication does not automatically replicate schema definitions or DDL commands. Publisher and subscriber table schemas must remain compatible for replicated changes. Sequence state is also not replicated, which must be considered if a subscriber is later made writable as part of a migration or role transition.

Official PostgreSQL source: [PostgreSQL 18 - Logical Replication Restrictions](#)

9.4 Logical Replication Failover

PostgreSQL 18 supports synchronization of logical failover slots to a physical standby so that logical subscribers can continue after the publisher standby is promoted. A subscription created with `failover = true` creates a slot eligible for this failover behavior.

Slot synchronization is asynchronous. PostgreSQL documentation requires confirming that failover slots are synchronized before failover; for a successful transition, the standby must be sufficiently advanced relative to the subscriber.

Official PostgreSQL source: [PostgreSQL 18 - Logical Replication Failover](#)

9.5 `sync_replication_slots` and `synchronized_standby_slots`

```
sync_replication_slots = on
```

On a physical standby, `sync_replication_slots` enables periodic synchronization of logical failover slots from the primary. PostgreSQL also provides `synchronized_standby_slots` on the primary to coordinate logical failover slot consumption with specified physical standby slots when required by the design.

Official PostgreSQL source: [PostgreSQL 18 - Replication Configuration](#)

10. Alternative HA Architectures Documented by PostgreSQL

10.1 Shared-Disk Failover

PostgreSQL documentation describes shared-disk failover as an architecture with one database copy on storage shared by multiple servers. If the active database server fails, another server can mount and start the same database. Because there is only one database copy, PostgreSQL documentation describes this as rapid failover with no replication-related data loss.

Major limitation: The shared disk array is a common failure domain. If it fails or becomes corrupt, both primary and standby servers can be nonfunctional. The standby must never access the shared storage while the primary is running.

Official PostgreSQL source: [PostgreSQL 18 - Comparison of Different Solutions](#)

10.2 File-System / Block Replication

PostgreSQL documentation also describes file-system replication, where changes are replicated below the database server at the storage or file-system layer. This is an external infrastructure technology rather than PostgreSQL physical streaming replication.

Official PostgreSQL source: [PostgreSQL 18 - Comparison of Different Solutions](#)

11. Monitoring and Operational Checks

11.1 Primary-Side Replication Monitoring

```
SELECT
    application_name,
    client_addr,
    state,
    sync_state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn
FROM pg_stat_replication;
```

pg_stat_replication reports one row for each WAL sender process, providing current connection and WAL-location information for directly connected standbys.

Official PostgreSQL source: [PostgreSQL 18 - pg_stat_replication](#)

11.2 Identify Primary vs Standby Recovery State

```
SELECT pg_is_in_recovery();
```

Result	Meaning
false	Server is not currently in recovery (normally a primary).
true	Server is currently in recovery (standby / recovery state).

Official PostgreSQL source: [PostgreSQL 18 - System Administration Functions](#)

11.3 Standby WAL Receive and Replay LSN

```
SELECT pg_last_wal_receive_lsn();
SELECT pg_last_wal_replay_lsn();
```

These functions report the last WAL location received through streaming replication and the last WAL location replayed during recovery, respectively.

Official PostgreSQL source: [PostgreSQL 18 - System Administration Functions](#)

11.4 Replication-Slot Operational Check

```
SELECT
    slot_name, slot_type, active, inactive_since,
    restart_lsn, confirmed_flush_lsn,
    wal_status, safe_wal_size, invalidation_reason
```

```
FROM pg_replication_slots
ORDER BY slot_name;
```

Review focus: Inactive slots, increasing WAL retention, wal_status changes, safe_wal_size, and invalidation_reason should be reviewed together with pg_wal filesystem capacity.

12. Failure Scenarios and Resiliency Model

12.1 Scenario - Primary Server Failure

```
Primary unavailable
|
External HA decision / fencing
|
Standby promotion
|
Client service redirected to new primary
```

PostgreSQL supplies promotion. The complete detection, fencing, and client-endpoint transition are responsibilities of the surrounding HA system.

12.2 Scenario - Site Failure

```
Production site unavailable
|
Remote physical standby available
|
Verify recovery / replay state
|
Promote selected DR standby
|
Redirect service
```

With asynchronous remote replication, the final recovery point is determined by the WAL actually received and replayed before the failure, not by a fixed time-based guarantee.

12.3 Scenario - Accidental DROP TABLE

```
Primary: DROP TABLE customer;
| WAL
v
Physical standby replays the same DROP
```

Physical replication reproduces database changes, including unwanted ones. A separate backup and WAL archive strategy is therefore required to recover to a point before the change.

12.4 Scenario - Replication Slot Causes WAL Growth

```

Consumer offline
  |
Slot remains
  |
Required WAL retained
  |
pg_wal grows
  |
Disk pressure / possible outage risk

```

PostgreSQL slot controls and `pg_replication_slots` monitoring should be used to detect and manage this condition.

12.5 Scenario - Hot Standby Query Conflict

```

Long read-only query on standby
  |
Primary WAL requires conflicting recovery action
  |
Recovery waits within configured delay
  |
Conflicting standby query may be canceled

```

The administrator must balance reporting-query duration, recovery delay, and the effects of `hot_standby_feedback`.

13. Production Design Principles

13.1 Layered Resiliency

PostgreSQL documentation presents HA replication and backup/recovery as complementary areas. A resilient design commonly combines separate mechanisms so that failure of one protection layer does not remove all recovery options.

```

Availability layer
  Physical standby / streaming replication / promotion

Recovery layer
  Base or incremental backup + archived WAL + PITR

Operational HA layer
  Failure detection + fencing + client routing (external to PostgreSQL)

```

13.2 HA Is Not a Backup

Core DBA principle: Physical replication protects availability by reproducing WAL changes. It also reproduces unwanted changes. Independent backup and PITR capability are required for recoverability from many logical/user-error scenarios.

Official PostgreSQL source: [PostgreSQL 18 - Backup and Restore](#)

13.3 Native PostgreSQL vs External HA Responsibilities

PostgreSQL 18 Provides	Complete HA System Must Also Provide
Physical streaming replication	Automatic failure detection / decision logic
Synchronous and asynchronous replication	Fencing / old-primary isolation
Warm and hot standby	Client endpoint / routing transition
Cascading replication	Cluster-wide service coordination
Replication slots	Operational alerting around failure conditions
Standby promotion	Controlled rejoin / failback procedure
WAL archiving and PITR	Surrounding infrastructure redundancy
Base and incremental backup utilities	Backup lifecycle policy and independent retention management
Logical replication and failover-slot support	Application-specific continuity planning

13.4 Operational Controls Checklist

1. Define the required business RPO and RTO before choosing a replication or recovery design.
2. Keep physical primary and standby release compatibility aligned with PostgreSQL requirements and recommendations.
3. Monitor streaming state and WAL positions through PostgreSQL statistics views.
4. Monitor replication slots and `pg_wal` capacity so inactive consumers do not create uncontrolled WAL retention.
5. If synchronous replication is used, document `synchronous_standby_names` and `synchronous_commit` because durability and availability behavior depend on both.
6. Test standby promotion and the surrounding failover procedure, including isolation of the old primary.
7. Test restore from physical backup and archived WAL, not only backup creation.
8. Test PITR to representative recovery targets and confirm required WAL retention.
9. For incremental backups, retain every predecessor backup required by the chain and validate restoration through `pg_combinebackup`.
10. For logical replication failover, verify slot synchronization before relying on publisher failover continuity.

Appendix A. Official PostgreSQL References

The following PostgreSQL Official Documentation pages are the exclusive technical source set used for this SOP. No third-party webpages, blogs, vendor manuals, or community articles are used as authoritative sources.

1. **PostgreSQL 18 Documentation Home** - <https://www.postgresql.org/docs/18/index.html>
2. **Chapter 25 - Backup and Restore** - <https://www.postgresql.org/docs/18/backup.html>
3. **25.3 Continuous Archiving and Point-in-Time Recovery (PITR)** - <https://www.postgresql.org/docs/18/continuous-archiving.html>
4. **pg_basebackup** - <https://www.postgresql.org/docs/18/app-pgbasebackup.html>
5. **pg_combinebackup** - <https://www.postgresql.org/docs/18/app-pgcombinebackup.html>
6. **Chapter 26 - High Availability, Load Balancing, and Replication** - <https://www.postgresql.org/docs/18/high-availability.html>
7. **26.1 Comparison of Different Solutions** - <https://www.postgresql.org/docs/18/different-replication-solutions.html>
8. **26.2 Log-Shipping Standby Servers** - <https://www.postgresql.org/docs/18/warm-standby.html>
9. **26.3 Failover** - <https://www.postgresql.org/docs/18/warm-standby-failover.html>
10. **26.4 Hot Standby** - <https://www.postgresql.org/docs/18/hot-standby.html>
11. **19.5 WAL Configuration** - <https://www.postgresql.org/docs/18/runtime-config-wal.html>
12. **19.6 Replication Configuration** - <https://www.postgresql.org/docs/18/runtime-config-replication.html>
13. **Recovery Configuration** - <https://www.postgresql.org/docs/18/recovery-config.html>
14. **Chapter 29 - Logical Replication** - <https://www.postgresql.org/docs/18/logical-replication.html>
15. **Logical Replication Restrictions** - <https://www.postgresql.org/docs/18/logical-replication-restrictions.html>
16. **29.3 Logical Replication Failover** - <https://www.postgresql.org/docs/18/logical-replication-failover.html>
17. **47.2 Logical Decoding Concepts / Slot Synchronization** - <https://www.postgresql.org/docs/18/logicaldecoding-explanation.html>
18. **pg_replication_slots View** - <https://www.postgresql.org/docs/18/view-pg-replication-slots.html>
19. **Monitoring Statistics / pg_stat_replication** - <https://www.postgresql.org/docs/18/monitoring-stats.html>
20. **System Administration Functions** - <https://www.postgresql.org/docs/18/functions-admin.html>

Document End

Controlled-source statement: This SOP is intentionally limited to PostgreSQL Official Documentation. Product-specific implementation guidance for external HA managers, proxies, storage systems, or cloud services is outside the source scope of this document.